A The Capacitated *k*-centroid Algorithm

We derive the decomposition structure by adopting a capacitated k-centroid method [5], which imposes an upper limit on the number of dimensions within each subgroup. The pseudocode of the Capacitated k-centroid is provided below, as shown in Algorithm 3.

Algorithm 3 Capacitated k-centroid

Input: Number of clusters K, capacity constraint C, distance matrix D $(n \times n)$, maximum iterations T.
Output: Cluster assignments clusters for n samples.
Initialize centroids using k-centroid++.
Initialize cluster sizes **sizes** as **0**.
for iteration $t = 1, 2, \ldots, T$ **do**
  Assignment Step:
  for each sample $i = 1, 2, \ldots, n$ **do**
    Compute distances from sample i to all centroids.
    Sort centroids in ascending order of distance.
    for each centroid k in sorted order **do**
      if sizes$[k] < C$ **then**
        Assign sample i to cluster k.
        Increment sizes$[k]$.
        break.
      end if
    end for
  end for
  Update Step:
  for each cluster $k = 1, 2, \ldots, K$ **do**
    Find samples assigned to cluster k.
    if cluster k is nonempty **then**
      Compute intra-cluster distances for cluster k.
      Set the new centroid as the point minimizing total intra-cluster distance.
    else
      Keep the current centroid unchanged.
    end if
  end for
  if centroids do not change **then**
    break.
  end if
end for
Return: clusters.

B Decomposition Accuracy Metrics

Here we introduce how we measure the discrepancy between a discovered decomposition structure and its target structure. Let S_{pred} and S_{target} denote the dis-

covered decomposition structure and the target structure, respectively. Each decomposition can be represented by a binary relationship matrix $A \in \{0,1\}^{D\times D}$, where $A_{ij} = 1$ indicates that dimensions i and j are dependent, and $A_{ij} = 0$ otherwise. Let A_{pred} and A_{true} denote the matrices induced by S_{pred} and S_{target}, respectively. The decomposition accuracy is calculated as

$$\rho_{\text{all}}(S_{\text{pred}}) = \frac{1}{D^2}\sum_{i,j}\mathbb{I}[A_{\text{pred}}(i,j) = A_{\text{true}}(i,j)]\,, \tag{1}$$

which counts the proportion of correctly identified dependent and independent pairs.

C Detailed Settings in Experiments

C.1 Synthetic Functions

To systematically examine our proposed methods, we conduct a comprehensive evaluation on three typical categories of synthetic functions: fully additive, fully non-additive, and moderately additive functions. Table 3 summarizes the test functions used in our experiments. For each function, we consider multiple dimensionalities: 40, 60, and 80. For the moderately additive functions (f1, f3, f5), the group structures are generated randomly, following the rule that each group contains 10 variables on average.

C.2 Implementation of baseline methods

We use the implementations referred from the original papers, for TuRBO [8], MCTS-VS [31], BOIDS [24], D_scaled [15] and RDUCB [41]. For the Tree [13], we directly use the implementation from RDUCB, where Tree is included as a baseline.

D Additional Experimental Results

D.1 Tree vs. RDUCB

Both RDUCB [41] and Tree [13] are based on tree-structured decompositions; Tree relies on likelihood-based modeling, whereas RDUCB uses random decompositions. We compare their optimization performance on synthetic functions and real-world problems in Table 4, Fig. 6 and Fig. 7. The results demonstrate that RDUCB is always superior to Tree except Walker2d, revealing that random decomposition can generally achieve superior performance to full-dimensional likelihood methods, which is consistent with the previous observations [41].

Table 3. Synthetic benchmark functions used in the experiments.

Function	Definition	Type
Ackley	$f(x) = -20\exp\Big(-0.2\sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2}\Big) - \exp\Big(\frac{1}{D}\sum_{i=1}^{D}\cos(2\pi x_i)\Big) + 20 + e$	Fully Non-Additive
f01	$f(x) = \Bigg(\sum_{i=1}^{\lfloor D/2\rfloor} x_i - \sum_{i=\lfloor D/2\rfloor+1}^{D} x_i\Bigg)^2$	Fully Non-Additive
f02	$f(x) = \Big(\sum_{i=1}^{D/2} x_i - \sum_{i=D/2+1}^{D} x_i\Big)^3$	Fully Non-Additive
Rastrigin	$f(x) = \sum_{i=1}^{D}\big(x_i^2 - 10\cos(2\pi x_i) + 10\big)$	Fully Additive
f03	$f(x) = \sum_{i=1}^{\lfloor D/2\rfloor} x_i - \sum_{i=\lfloor D/2\rfloor+1}^{D} x_i$	Fully Additive
f04	$f(x) = \sum_{i=1}^{\lfloor D/2\rfloor} x_i^2 - \sum_{i=\lfloor D/2\rfloor+1}^{D} x_i^2$	Fully Additive
f1	$f(x) = \sum_{g=1}^{G}\prod_{i\in\mathcal{G}_g} x_i$	Moderately Additive
f3	$f(x) = \sum_{g=1}^{G}\Big(\sum_{i\in\mathcal{G}_g} x_i\Big)^2$	Moderately Additive
f5	$f(x) = \sum_{g=1}^{G}\Big(\sum_{i\in\mathcal{G}_g} x_i\Big)^3$	Moderately Additive

D.2 Effectiveness of Pairwise Metrics

To demonstrate the effectiveness of the proposed metrics, we compare them with the random metric under the same decomposition setting (denoted as PRBD_Random_TuRBO). The results are shown in Fig. 8. We can observe that all of our metrics can achieve superior performance to random, demonstrating their effectiveness.

Table 4. Comparison the optimal y between RDUCB and Tree on synthetic function.

Function	**Tree**	**RDUCB**
Ackley (D=80)	-11.991 ± 0.440	$\mathbf{-11.883 \pm 0.429}$
Rastrigin (D=80)	-997.756 ± 21.529	$\mathbf{-967.327 \pm 50.583}$
f3 (D=80)	$\mathbf{9974.692 \pm 971.019}$	9385.939 ± 990.051
f5 (D=80)	$2.9e5 \pm 4.9e3$	$\mathbf{3.5e5 \pm 2.4e4}$

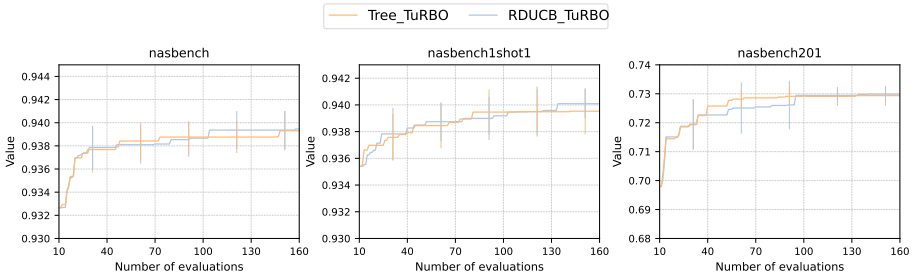

Fig. 6. Performance comparison of Tree and RDUCB on NAS-Bench problems.

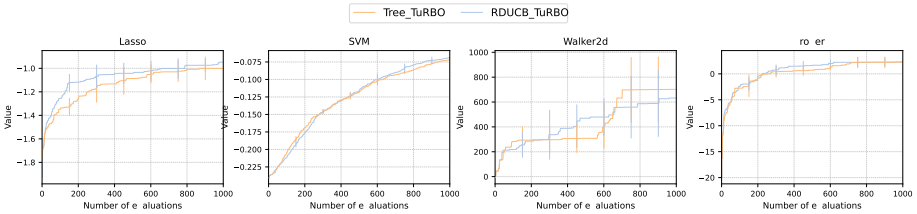

Fig. 7. Performance comparison of Tree and RDUCB on real-world problems.

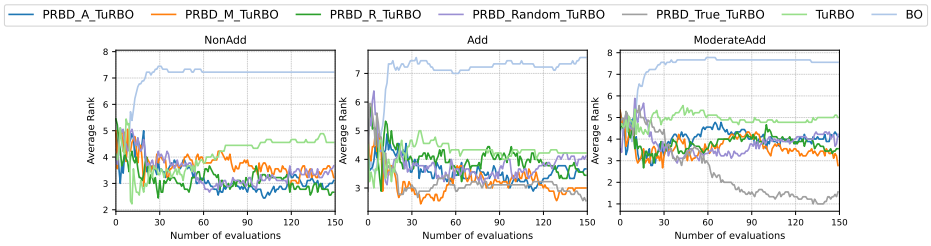

Fig. 8. Comparison on average rank among different PRBD variants.

D.3 Sensitivity Analysis of C

Capacitated k-centroid clustering (Algorithm 3) is applied to decompose with the capacity constraint C and the number of groups is set as $\lceil D/C \rceil$, as introduced in Section 5.1 of the main paper. Here are the complete results on synthetic functions, which are illustrated in Fig. 9. Specifically, we illustrate the results on f1 (moderately additive), Rastrigin (fully additive), and Ackley (fully non-additive) with $D = 80$ and $C \in \{10, 20, 40\}$.

Moderately Additive: on the function f1, the number of groups in the true decomposition is 10, and the variant with true decomposition achieves the best

performance as expected. But for other variants of PRBD, surprisingly, we can observe that the setting of $C = 20$ achieves the best average performance. A possible reason is that as the true decomposition can not be captured directly, a moderately larger C ($C = 20$) can help decompose the related dimensions in the same group. However, an extremely larger C ($C = 40$) leads to degraded performance due to the curse of dimensionality.

Fully Additive: on the function Rastrigin, the performance of different variants is very close. As the true decomposition corresponds to $C = 1$, in principle, this setting should be optimal, as it exactly matches the underlying structure. However, the experimental results indicate that $C = 1$ performs slightly worse than moderately larger values of C. This can be attributed to the fact that $C = 1$ requires learning D independent scaling parameters in GP, whereas increasing C reduces the number of parameters to be estimated without violating variable independence, thereby improving the model stability.

Fully non-additive: on the function Ackley, the performance of different variants is very close as well. Applying the true decomposition where $C = D$ achieves marginally better performance, followed by $C = 40$, which is close to the true decomposition. Nevertheless, in high-dimensional settings, excessively large group sizes can potentially exacerbate the curse of dimensionality, which in turn leads to degraded optimization performance.

Overall, these results suggest that the performance of additive-based BO methods is jointly influenced by the accuracy of the decomposition, the dimensionality of the sub-problems, and the intrinsic complexity of the objective function. In practice, a moderate value of C is suggested across different problem types.

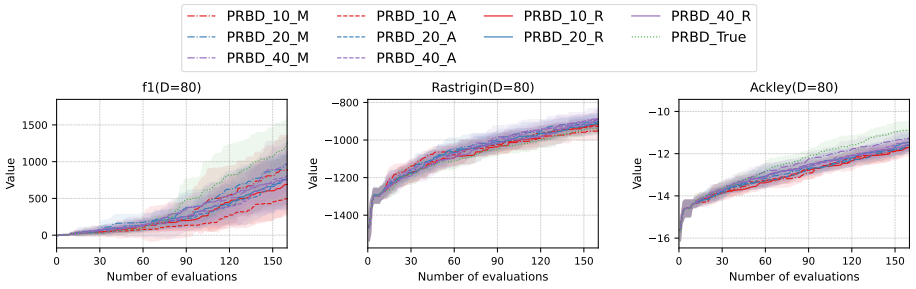

Fig. 9. Sensitivity analysis of the C.

D.4 The Optimization Strategies for Acquisition Function

In decomposition-based Bayesian optimization, there are two optimization strategies: one where all subgroups are optimized jointly (as in this work), and another

one where each subgroup is optimized individually as shown in Algorithm 4. The experimental comparison is provided in Table 5. The results show that optimizing each subgroup individually degrades severely, possibly because it is more likely to get stuck in local optima compared to joint optimization. Moreover, joint optimization allows additive-based methods to be equipped with other advanced BO variants.

Algorithm 4 Add BO

Input: Kernels $k^{(1)}, \ldots, k^{(M)}$, decomposition $(\mathcal{G}^{(j)})_{j=1}^{M}$.
Output: Optimal solution $\mathbf{x}^*, y^*$
Initialize $\mathcal{D}_0$.
for iteration $t = 1, 2, \ldots$ **do**
  for each $j = 1, \ldots, M$ **do**
    $\mathbf{x}_t^{(j)} \leftarrow \arg\max_{z \in \mathcal{X}^{(j)}} \left(AF^{(j)} \right)$.
  end for
  $\mathbf{x}_t \leftarrow \bigcup_{j=1}^{M} \mathbf{x}_t^{(j)}$.
  $y_t \leftarrow$ Query f at $\mathbf{x}_t$.
  Update $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\mathbf{x}_t, y_t)\}$.
end for

Table 5. Best observed y. Large numbers are shown in scientific notation for clarity.

Function	PRBD_Random_group	PRBD_Random_TuRBO	PRBD_True_group	PRBD_True_TuRBO
Ackley40	-13.700 ± 0.219	**-9.958 ± 0.606**	-13.718 ± 0.103	**-9.728 ± 0.399**
Ackley60	-14.118 ± 0.175	**-11.403 ± 0.347**	-13.949 ± 0.151	**-10.791 ± 0.167**
Ackley80	-14.340 ± 0.133	**-11.659 ± 0.300**	-14.207 ± 0.063	**-11.083 ± 0.481**
Rastrigin40	-5.83e2 ± 1.23e1	**-4.13e2 ± 2.87e1**	-6.65e2 ± 2.21e1	**-4.16e2 ± 3.28e1**
Rastrigin60	-8.84e2 ± 2.17e1	**-6.64e2 ± 4.30e1**	-9.54e2 ± 4.39e1	**-7.00e2 ± 3.86e1**
Rastrigin80	-1.24e3 ± 4.54e1	**-9.48e2 ± 2.04e1**	-1.29e3 ± 2.98e1	**-9.43e2 ± 2.08e1**
f340	1.95e3 ± 7.50e2	**5.50e3 ± 312**	2.36e3 ± 5.16e2	**5.58e3 ± 339**
f360	2.01e3 ± 322	**6.96e3 ± 738**	2.27e3 ± 4.79e2	**7.33e3 ± 765**
f380	2.21e3 ± 252	**7.57e3 ± 1.50e3**	2.73e3 ± 744	**8.50e3 ± 782**

D.5 Equip with D_scaled

It has been discussed that our proposed PRBD can be easily equipped with variants of BO, as in Section 5.1. Here we provide the results on another function f3 ($D = 40$), as shown in Fig. 10. We can observe that integrating PRBD can bring improvements. Note that a non-trivial variance can be observed. A possible reason is that the backbone method D_scaled tends to enhance exploration during the optimization.

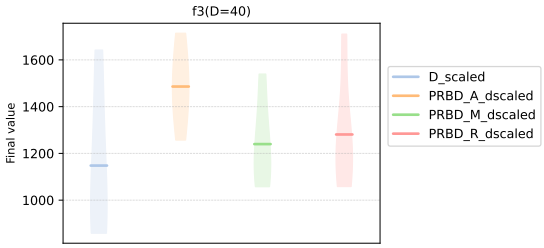

Fig. 10. Distribution of optimal values for different algorithms on the synthetic function after 160 iterations.

D.6 Comparison among Different Metrics

We analyze the computational complexity of the three interaction metrics considered in this work. In each iteration, the **Average Goodness** updates the interaction values for each pair of dimensions within the same group, resulting in a complexity of $O(d^2)$. The **Pairwise Marginal Likelihood** computes the log marginal likelihood for each pair of dimensions by fitting two-dimensional GPs, resulting in a complexity of $O(d^2n^3)$. Finally, the **Polynomial Regression** constructs a linear regression model with pairwise interaction terms, estimates the strength of pairwise interaction directly via the absolute values of coefficients, with a complexity of $O(nd^4)$.

To demonstrate that PRBD is generally efficient, we report the runtime of variants of PRBD, Tree_TuRBO and TuRBO in Table 6. As runtime on different functions can be totally different, here we take f1 with $D = 60$ as an example. We can observe that equipping TuRBO with PRBD leads to approximately $2 \sim 3$ times increase on runtime, but a large part of it comes from the framework of additive BO, as PRBD_Random_TuRBO still leads to approximately 2 times increase on runtime. PRBD_M_TuRBO suffers the longest runtime among the variants of PRBD, demonstrating that its superior performance in experiments comes at the cost of additional runtime. Nevertheless, it is worth noting that for PRBD_M_TuRBO the calculation of each pair is independent, which can be easily parallelized for acceleration in future works. Meanwhile, we can observe that our PRBD has great advantages on runtime against Tree_TuRBO.

Table 6. Average runtime comparison (f1 with $D = 60$)

Algorithm	Time
PRBD_A_TuRBO	327 s
PRBD_M_TuRBO	532 s
PRBD_R_TuRBO	339 s
PRBD_Random_TuRBO	326 s
Tree_TuRBO	5616 s
TuRBO	118 s